

Design Your Own Generative AI Application

Team size	4–6 students
Report submission deadline	13 September 2026, 11:59pm
Group presentation	During class in Week 6
Main goal	Design, prototype, test, and improve a GenAI application using your own system decisions

Core rule: AI tools may implement your design, but they must not replace your design decisions.

1. Assignment Objective

Your task: Identify a real-world problem and design, prototype, and evaluate a Generative AI application that addresses it.

The project is not about writing code. It is about making sound GenAI system decisions: what the AI should do, what information it should use, how prompts should control each AI module, how the application should be evaluated, and where human judgement is needed.

2. Assignment Scope

Use existing tools to reduce implementation effort, not thinking effort.

YOUR TEAM MUST DESIGN	TOOLS MAY HELP IMPLEMENT
Problem, target users, and success criteria	UI layout, forms, buttons, navigation
System architecture and module boundaries	Exact workflow implementation
The purpose, input, and output of each AI module	Generated implementation code behind the interface
Prompt(s) for each AI module	Visual styling and presentation
Test cases, evaluation criteria, and failure analysis	Rapid revisions to the prototype

Examples of implementation tools: Gemini Canvas or other accessible no-code / AI-assisted app builders. You may ask the tool to build the interface and implement your specified workflow. You cannot simply ask it to decide the application architecture, prompts, knowledge strategy, and evaluation for you.

3. Mandatory Project Pipeline

Follow the stages below in order. Your final submission must provide evidence for every stage.

Stage 1 — Define the Problem

- Choose one focused problem for one clearly defined user group; state the user need and desired outcome.
- Define 2–4 success criteria before building anything.

Stage 2 — Design the System Architecture

- Draw the information flow from user input to final output and identify the major modules.
- Explain the functionality for each module within the information flow, such as UI / retrieval / LLM / multimodal model / human review.

Stage 3 — Specify the AI Modules and Prompts

- Include at least two AI-enabled modules or behaviours with different purposes. Name which AI models you are using.
- For each, define inputs, exact prompt/instructions, context, and output format.
- Briefly justify the important prompt instructions.

Stage 4 — In-Context Learning / Lightweight RAG

- Identify which outputs require in-context examples (few-shot prompting), or external evidence (retrieval). Retrieval is optional, but including it meaningfully will earn extra credit.
- If you are using in-context learning, create relevant examples in the prompt and justify the importance of your design.
- If you are using retrieval-augmented generation, create a small, curated knowledge collection (suggested: 10–30 items/pages/snippets/webpages) for retrieval. Define how evidence is selected (i.e., how retrieval is performed): manual, rule-based (e.g., token matching), prompt-based, or others.

Stage 5 — Build the Prototype

- Now use Gemini Canvas or another relevant AI builder to implement the UI and workflow you already designed. The prototype should make the major modules visible in the UI.
- Name the AI builders you used and what functions they provided in the system.

Stage 6 — Evaluate the System

- Create 20 test cases before final evaluation. Make sure the test cases contain normal and challenging scenarios such as ambiguous query, missing-information, misleading instruction.
- Compare at least three variants on the same test set: (A) minimal LLM, (B) simplified system (without your advanced design), and (C) your fully designed system. List the comparison results in the final report.
- Define 4 evaluation criteria appropriate to your application (e.g., correctness, grounding, clarity, personalization, safety, creativity). If appropriate, use an LLM as the judge.

Stage 7 — Analyze Failures and Improve

- Identify failure cases.
- For each failure case, document: input → expected behaviour → actual behaviour → likely cause → proposed fix → retest result.
- Explain which improvements came from better prompts, better few-shot examples/retrieval, better workflow design, or others.

4. Mini Worked Example: AI CourseMate

This example illustrates the design and workflow. You are not allowed to copy or reuse this example.

Problem	Students need explanations that are appropriate to their level and grounded in course material.
Module 1	Retrieval: select 1–2 relevant course-note entries for the student's question.
Module 2	Explanation generator (Prompt A): answer using retrieved evidence; adapt to chosen difficulty; state when evidence is insufficient.
Module 3	Quiz generator (Prompt B): create one conceptual MCQ from the same evidence and reveal feedback after the student answers.
Prototype	Canvas implements the interface: question box, difficulty selector, Explain button, Quiz button, and output panels.
Evaluation	Compare plain LLM vs. Prompt A only vs. retrieval + Prompt A on the same test questions.

Example flow: Student question → retrieve relevant notes → Prompt A → grounded explanation → Prompt B → quiz + feedback.

5. Required Deliverables

1. A report explaining each stage above. Your report should be within 10 pages and in pdf format. Include a cover page listing team members (name and email), together with the contribution of each team member. The cover page does not count towards the 10-page limit.
2. A link to access the Generative AI system your team has built. Append the link to the end of your report. Make sure the link can work successfully.
3. Final presentation in class with demos.

6. In-class Final Group Presentation (<=10 mins)

- 6 minutes: problem, architecture, and evaluation.
- 2 minutes: live or recorded demonstration.
- 2 minutes: Q&A

7. Assessment Rubric

Component	Weight	Evidence expected
Problem formulation & creativity	10%	Focused user need; appropriate use of GenAI
System architecture & module design	20%	Clear decomposition; justified information flow and module roles
Prompt & AI modules	20%	Well-specified prompts; meaningful and justified AI modules
Evaluation & controlled comparison	10%	Good test set; appropriate metrics; fair comparison of variants
Failure analysis & improvement	10%	Insightful diagnosis, fixes, and retesting
Presentation	30%	Clear explanation of what the team designed, what tools implemented, and why the system is meaningful